

Лабораторная работа №7

Экран списка объявлений: RecyclerView и CardView

О чем лабораторная?

В предыдущем уроке вы разобрали, как **RecyclerView** выводит большой список с помощью адаптера и **ViewHolder**. Теперь добавим в приложение основной экран с локальным списком объявлений, а каждое объявление оформим отдельной карточкой.

Главное, что нужно понять

- **RecyclerView** отображает данные из **List<Item>**
 - **Adapter** и **ViewHolder** связывают объект объявления с его разметкой
 - **CardView** оформляет один элемент списка как карточку
 - модель, разметка элемента и адаптер - разные части одной задачи
-

Часть 1. Подготовьте экран объявлений

Создайте или измените главный экран приложения так, чтобы после OnBoard открывался список объявлений. В **activity_main.xml** добавьте **RecyclerView**:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rv_items"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="8dp"
    android:clipToPadding="false" />
```

Часть 2. Создайте модель объявления

Создайте класс **Item**. Пока список работает с локальными демонстрационными данными - позже вместо локального списка мы будем загружать объявления с учебного сервера (FastAPI) и показывать изображения.

```
public class Item {
    private final String title;
    private final String price;
    private final String description;

    public Item(String title, String price, String description) {
        this.title = title;
        this.price = price;
        this.description = description;
    }

    public String getTitle() {
```

```

        return title;
    }

    public String getPrice() {
        return price;
    }

    public String getDescription() {
        return description;
    }
}

```

Названия полей совпадают с данными, которые будут приходить с сервера, - так позже будет проще подключить загрузку объявлений.

Часть 3. Создайте карточку объявления

Создайте файл `res/layout/row_item.xml`. Внутри `CardView` расположите вертикальный `LinearLayout` и три `TextView`: для названия, цены и описания.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="3dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- здесь создайте само содержимое карточки (не забудьте добавить id) -->

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

`CardView` не хранит данные и не создает список. Его задача только оформить **одну** строку `RecyclerView`

Часть 4. Самостоятельная работа: адаптер

Создайте `ItemAdapter`, наследующийся от `RecyclerView.Adapter`. Используйте структуру с урока №6: адаптер принимает `List<Item>`, создает `ItemViewHolder` и заполняет карточку в `onBindViewHolder()`

Если что-то забыли, откройте урок №6

Минимальные каркасы адаптера и ViewHolder:

```
static class ItemViewHolder extends RecyclerView.ViewHolder {  
    // Добавьте поля для сохранения TextView  
  
    public ItemViewHolder(@NonNull View itemView) {  
        super(itemView);  
  
        // Найдите ваши TextView по id  
    }  
}
```

```
public class ItemAdapter extends RecyclerView.Adapter<ItemViewHolder> {  
  
    private final List<Item> items;  
  
    // обязательно создаем конструктор, который принимает список для адаптера  
    public ItemAdapter(List<Item> items) {  
        this.items = items;  
    }  
  
    // самостоятельно реализуйте:  
    //  
    // onCreateViewHolder()  
    // onBindViewHolder()  
    // getItemCount()  
}
```

Проверьте себя до запуска:

1. В `onCreateViewHolder()` используется `R.layout.item_item`.
2. В `onBindViewHolder()` объект берется из `items` по `position`.
3. Название, цена и описание выводятся в соответствующие `TextView`.
4. `getItemCount()` возвращает размер списка.

Часть 5. Подключите список в MainActivity

В `MainActivity` создайте несколько объектов `Item`, настройте вертикальный `LinearLayoutManager` и адаптер, а затем подвяжите их к вашему `RecyclerView`. Не копируйте строки с экрана: придумайте не меньше пяти разных объявлений.

Пример набора данных:

```
List<Item> items = new ArrayList<>();  
items.add(new Item("Велосипед", "12 000 Р", "Городской велосипед в хорошем состоянии"));
```

```
items.add(new Item("Учебник Java", "700 ₽", "Книга для начинающих разработчиков"));
items.add(new Item("Настольная лампа", "1 500 ₽", "Светодиодная лампа для рабочего
стола"));
```

Часть 6. Проверьте результат

1. Откройте приложение после прохождения OnBoard
2. Убедитесь, что главный экран показывает список минимум из пяти объявлений
3. Прокрутите список: объявления должны отображаться без ошибок
4. Проверьте, что у каждого объявления есть название, цена и описание
5. Убедитесь, что каждый элемент выглядит как отдельная карточка

Базовый уровень

Создайте экран со списком минимум из пяти локальных объявлений. Для каждого объявления должны выводиться название, цена и описание в карточке `CardView`

Дополнительно

1. Добавьте обработку нажатия на карточку: выводите `Toast` с названием выбранного объявления

Требования к итоговой версии

1. Главный экран содержит `RecyclerView` со списком объявлений
2. Список использует `LinearLayoutManager` и собственный `ItemAdapter`
3. Созданы модель `Item`, разметка `item_item.xml` и `ViewHolder`
4. Каждый элемент отображается в `CardView` и показывает название, цену и описание
5. В списке есть минимум пять разных локальных объявлений
6. Все основные строки интерфейса вынесены в `strings.xml`